# Point Operations & Histograms

Week 2 — Fixing exposure and contrast

Practical Image Processing

Lecture 2 ▪ 60–70 min

# Recap & today

- Wk1: an image is numbers on a grid; watch BGR, uint8, [y,x].
- Today: **point operations** — transform each pixel by itself — and the **histogram**, the tool that tells you what a transform should do.

**Why it matters**

DocScan's meter photos are often too dark or low-contrast. Fixing exposure is step one to reading the digits.

# Roadmap

The histogram

Point operations

Contrast enhancement

In-class exercise

Thresholding

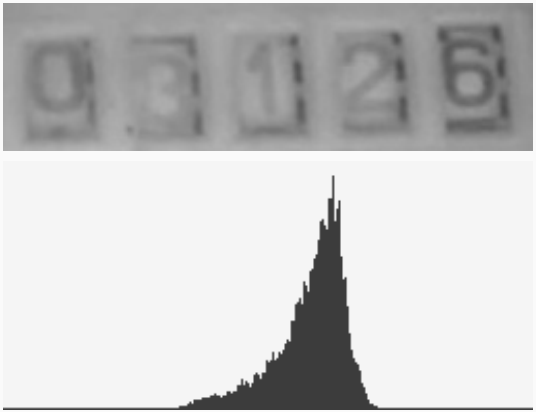
Deep dive

Wrap-up

# The histogram

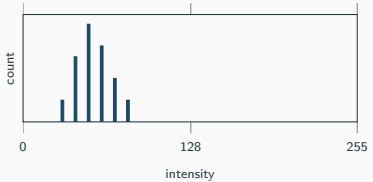
# The histogram: a census of brightness

A real meter crop and its histogram:



- $x$-axis: intensity 0–255. $y$-axis: how many pixels have it.
- This histogram is bunched $\rightarrow$ **low contrast** (digits barely stand out).
- Read the histogram *before* touching the image.

# Reading a histogram




- Peak on the **left** = dark (under-exposed). On the **right** = bright.
- **Narrow** = low contrast. **Spread across the range** = good contrast.
- Two peaks (**bimodal**) = object vs background — a gift for thresholding (later today).

# Computing a histogram

```python
import cv2, numpy as np
gray = cv2.cvtColor(cv2.imread('E:/data/abfa_image/numerators/num_3494.jpg'),
                    cv2.COLOR_BGR2GRAY)
hist = cv2.calcHist([gray], [0], None, [256], [0, 256])  # 256 bins
print(gray.min(), gray.max(), gray.mean())   # quick exposure check
```

**Key idea**

The histogram throws away *where* pixels are and keeps *how many* of each value. That's exactly what you need to reason about exposure and contrast.

# Point operations

# Point operations: one pixel at a time

- A **point operation** maps each pixel's value through a function $s = T(r)$ — independent of neighbours.
- Brightness: $s = r + b$. Contrast: $s = a \cdot r$. Negative: $s = 255 - r$.
- Implemented as a **lookup table** (LUT): compute $T$ once for 0–255, apply to all pixels fast.

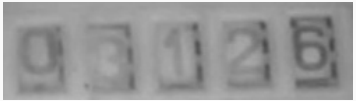
**Math**

General linear: $s = a \cdot r + b$ — $a$ scales contrast, $b$ shifts brightness. Then clip to 0–255.

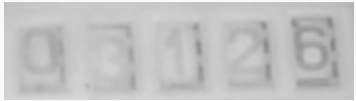
# Gamma correction — on the real meter



$\gamma < 1$ (darker)



original



$\gamma > 1$ (brighter)

**Math**

$s = 255\,(r/255)^{1/\gamma}$ — a *nonlinear* curve that brightens shadows without blowing out highlights.

# Brightness/contrast vs gamma

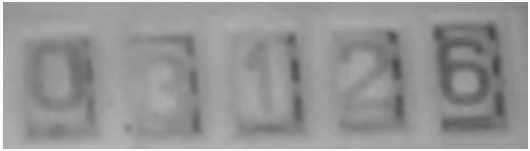
- Linear ($ar + b$) shifts/scales everything equally — can clip highlights or crush shadows.
- Gamma is **nonlinear**: it lifts dark tones more than bright ones (or vice versa).
- For a dark meter photo, gamma often recovers digits that a linear brighten would wash out.

**✔ Good practice**

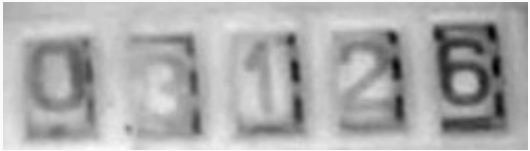
Match the tool to the histogram: shifted-but-spread → brightness; bunched → contrast/gamma.

# Contrast enhancement

# Contrast stretching (min–max)



before



after: stretched to 0–255

- Map the actual $[\min, \max]$ to the full $[0, 255]$.
- Simple, reversible, and it can't invent detail — just uses the full range.

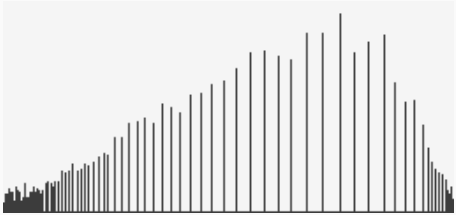
**Math**

$$s = 255 \frac{r - \min}{\max - \min}$$

# Histogram equalization



equalized meter



its histogram — spread out

- Redistributes intensities so the histogram is (roughly) **uniform**.
- Uses the **cumulative histogram** (CDF) as the mapping function.
- Great for revealing detail in low-contrast images — but can over-amplify noise.

# The three, in code

```
stretch = cv2.normalize(gray, None, 0, 255, cv2.NORM_MINMAX)
equalized = cv2.equalizeHist(gray)
lut = np.array([((i/255)**(1/0.5))*255 for i in range(256)], np.uint8)
gamma = cv2.LUT(gray, lut)                  # gamma = 0.5
```

**Note**

`cv2.LUT` applies a 256-entry table to every pixel in one fast pass — the pattern for any point op.

# In-class exercise

# In-class exercise (10 min)

**Diagnose from the histogram**

You are handed three meter photos. Their histograms are:

1. a tall spike near 0, nothing above 80;
2. two separated humps, one near 40 and one near 210;
3. a smooth spread from 0 to 255.

For each: is it under/over-exposed, low-contrast, or fine? Which operation would you apply, and which needs *no* enhancement? Which one is easiest to threshold, and why?

# Debrief

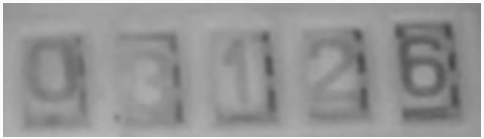
- (1) Under-exposed & low-contrast $\rightarrow$ gamma ($\gamma < 1$) or stretch; digits hide in the dark.
- (2) Bimodal $\rightarrow$ already separable; *ideal* for thresholding (object vs background).
- (3) Well-exposed, full range $\rightarrow$ leave it; enhancing may only amplify noise.

**Key idea**

The histogram tells you *whether* and *which* enhancement — before you try anything.

# Thresholding

# Thresholding: intensity $\rightarrow$ two classes



grayscale meter



Otsu binary: digits vs background

- $s = 255$ if $r > T$ else 0. Turns gray into black/white.
- The whole game is choosing $T$. A bimodal histogram makes it easy.
- This is DocScan's "make it crisp" step — digits become a clean mask.

# Otsu: pick the threshold automatically

- Otsu's method finds the $T$ that best **separates** the two histogram humps.
- Formally: minimize the within-class variance (equivalently, maximize between-class variance).
- One line: `cv2.threshold(g, 0, 255, THRESH_BINARY+THRESH_OTSU)`.

**✗ Pitfall**

Otsu assumes two clear classes. On uneven lighting (shadow across the page) a single global $T$ fails — use *adaptive* thresholding.

# Adaptive thresholding for uneven light

```
# a local threshold per neighbourhood -> robust to shadows/gradients
binary = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                               cv2.THRESH_BINARY, blockSize=31, C=10)
```

**Key idea**

Global Otsu for even lighting; adaptive for real photos with shadows. DocScan photos usually need adaptive.

# Deep dive

# CLAHE: local, limited equalization

- Global equalization over-amplifies noise in flat regions.
- **CLAHE** equalizes in small tiles and **clips** the histogram to limit amplification.
- Usually the better default for real photos: `cv2.createCLAHE(clipLimit=2.0)`.

**✓ Good practice**

For DocScan, CLAHE on the value channel often beats plain equalization — local contrast without the noise blow-up.

# Why equalization can hurt

- It's data-driven and **unpredictable**: two similar images can map very differently.
- It amplifies whatever is there — including sensor noise and JPEG blocks.
- For a pipeline, prefer a **deterministic** op (fixed gamma/stretch) so later stages are stable.

**Key idea**

"Looks better to the eye" $\neq$ "better for the next algorithm." Enhance for the goal, not the demo.

# Histogram matching

- Sometimes you want an image's histogram to *match* a reference (consistent look across a dataset).
- Useful when meter photos come from many cameras/lightings and you want uniform input.
- Map via the two CDFs — the same machinery as equalization, but to a target shape.

# Color: enhance value, not each channel

- Equalizing R, G, B *separately* shifts colors (ugly casts).
- Convert to HSV/Lab, enhance the **value/lightness** channel, convert back.
- Keeps hue intact while fixing brightness/contrast — the correct way to enhance color photos.

**✕ Pitfall**

Per-channel contrast on RGB is a classic beginner mistake that tints the whole image.

# Thresholding is a decision, not a default

- Below $T$ vs above $T$ is a **classification** — errors here propagate to every later step.
- Too high $T$: digits break up. Too low: background bleeds in. Tune on *hard* images.
- Combine: enhance (gamma/CLAHE) $\rightarrow$ then threshold (adaptive) $\rightarrow$ then clean (Week 7 morphology).

# Wrap-up

# What you should now be able to do

- Read a histogram to diagnose exposure and contrast.
- Apply brightness, contrast, gamma, stretching, equalization — and know which fits.
- Threshold with a global (Otsu) or adaptive method, and know when each fails.
- Enhance color correctly (value channel, not per-RGB).

# This week's deliverable

**Due before Week 3 (see `project/`)**

On real meter/plate images from `E:/data`:

1. Plot the histogram; classify each image's exposure/contrast.
2. Apply and compare: stretch, equalization, gamma, and CLAHE — with before/after.
3. Produce a clean Otsu *and* adaptive binary of the digits; say which wins and why.
4. Note: for your DocScan image, which enhancement + threshold combo reads best?

## Look at the histogram first.

**It tells you what the image needs before you change a single pixel.**